

```
cmake --build build/  
./build/Debug/task04.exe
```

Aufgabe04

- <https://docs.gl/>
-

4.1) Hinzufügen von Texturkoordinaten zu einem Quad

Weisen Sie ihren Quad-Vertices Texturkoordinaten zu, sodass die vier Eckpunkte eines quadratischen Bildes den Vertices zugeordnet werden.

Implementierung

- Texturkoordinaten so erstellen, dass die UV-Koordinaten von oben links (0,0) nach unten rechts (1,1) verlaufen. Wenn (0,0) oben links ist, dann steht die Textur auf dem Kopf.
- Der Unterschied kommt daher, da Bildformate (PNG, JPEG) die Pixel von oben links nach unten rechts speichern, während OpenGL die UV-Koordinaten standardmäßig von unten links nach oben rechts interpretiert
 - Man kann die Texturkoordinaten anpassen oder beim Laden des Bildes einen Flip durchführen, um die Textur korrekt darzustellen.

4.2) Erstellen einer OpenGL Textur auf der GPU

Finden Sie heraus, was die einzelnen Befehle bewirken und nehmen Sie die Erkenntnisse in Ihre Dokumentation mit auf.

```
void glCreateTextures(  
    GLenum target,  
    GLsizei n,  
    GLuint *textures  
);
```

- target: Der Typ der zu erzeugenden Textur-Objekte (z.B. GL_TEXTURE_2D).
- n: Anzahl der zu erzeugenden Textur-Objekte.
- *textures: Zeiger auf ein Array, in dem die Namen (Ids, wie VBO/VAO oder Handlers) der neuen Textur-Objekte gespeichert werden.

Wird verwendet, um ein oder mehrere Textur-Objekte als Speicherbereiche auf der GPU zu erzeugen.

```
void glTextureParameteri(  
    GLuint texture,  
    GLenum pname,  
    GLint param  
);
```

- texture: Name/ ID vom Textur-Objekt.
- pname: Parameter, der gesetzt werden soll (z.B. GL_TEXTURE_WRAP_S, GL_TEXTURE_MIN_FILTER).
- param: Wert, der für den angegebenen Parameter gesetzt werden soll (z.B. GL_CLAMP_TO_EDGE, GL_NEAREST).

Wird verwendet, um Parameter für ein bereits existierendes Textur-Objekt zu setzen.

```
glTextureParameteri(textureHandle, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
```

- Wrap-S -> S steht für die horizontale UV-Koordinate.

- Definiert den Fall, wenn UV-Koordinaten außerhalb des Bereichs [0, 1] liegen.
GL_CLAMP_TO_EDGE sorgt dafür, dass für alle UV-Koordinaten außerhalb des Bereichs die äußerste Randfarbe der Textur verwendet wird.

```
glTextureParameteri(textureHandle, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```

- Wrap-T -> T steht für die vertikale UV-Koordinate.
- Definiert den Fall, wenn UV-Koordinaten außerhalb des Bereichs [0, 1] liegen.
GL_CLAMP_TO_EDGE sorgt dafür, dass für alle UV-Koordinaten außerhalb des Bereichs die äußerste Randfarbe der Textur verwendet wird.

```
glTextureParameteri(textureHandle, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

- Min-Filter -> Definiert die Filtermethode, die verwendet wird, wenn die Textur verkleinert dargestellt wird (d.h. wenn mehrere Texel auf einen Pixel abgebildet werden). GL_NEAREST sorgt dafür, dass der nächstgelegene Texel-Wert verwendet wird.

```
glTextureParameteri(textureHandle, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
```

- Mag-Filter -> Definiert die Filtermethode, die verwendet wird, wenn die Textur vergrößert dargestellt wird (d.h. wenn ein Texel auf mehrere Pixel abgebildet wird). GL_NEAREST sorgt dafür, dass der nächstgelegene Texel-Wert verwendet wird.

```
void glTextureStorage2D(
    GLenum target,
    GLsizei levels,
    GLenum internalformat,
    GLsizei width,
    GLsizei height
);
```

- target: Der Typ der Textur (z.B. GL_TEXTURE_2D).
- levels: Anzahl der Mipmap-Level, die für die Textur reserviert werden sollen (muss mindestens 1 sein; 1 für nur die Basis-Textur ohne weitere Mipmaps, >1 für mehrere Mipmap-Level).
- internalformat: Das interne Format der Textur (z.B. GL_RGBA8, GL_RGB8).
- width: Breite der Textur in Pixeln.
- height: Höhe der Textur in Pixeln.

Reserviert unveränderlichen Speicher auf der GPU für die Textur und legt Format, Auflösung und Anzahl der Mipmap-Level fest. Es werden noch keine Pixeldaten hochgeladen. Mipmaps sind vorberechnete, verkleinerte Versionen der Textur, die abhängig von der Entfernung zur Kamera geladen werden. Wenn die Kamera weit entfernt ist, wird eine kleinere, weniger detaillierte Mipmap-Version der Textur verwendet, um die Leistung zu verbessern.

```
void glTextureSubImage2D(
    GLuint texture,
    GLint level,
    GLint xoffset,
    GLint yoffset,
    GLsizei width,
    GLsizei height,
    GLenum format,
    GLenum type,
    const void *pixels
);
```

- texture: Name/ ID vom Textur-Objekt.
- level: Mipmap-Level der Textur, die aktualisiert werden soll (0 für die Basis-Textur).

- `xoffset`, `yoffset`: Offset in Pixeln, der angibt, wo die Aktualisierung innerhalb der Textur beginnen soll.
- `width`, `height`: Breite und Höhe des Bereichs, der aktualisiert werden soll, in Pixeln.
- `format`: Format der Pixel-Daten (z.B. `GL_RGBA`, `GL_RGB`).
- `type`: Typ der Pixel-Daten (z.B. `GL_UNSIGNED_BYTE`).
- `*pixels`: Zeiger auf die Pixel-Daten.

Lädt Pixeldaten von der CPU in den zuvor reservierten GPU-Speicher. Ermöglicht das Aktualisieren eines Teilbereichs der Textur über `xoffset/yoffset`, ohne die gesamte Textur neu zu laden.

4.3) Zugänglich machen der Texturkoordinaten im Shader

‘Verdrahten’ Sie nun das Texturkoordinaten-Attribut über das VAO mit dem Vertex-Shader, sodass die Texturkoordinaten in diesem verfügbar werden. Ermöglichen Sie außerdem die Weiterleitung der Texturkoordinaten aus dem Vertex- in den Fragment-Shader.

Implementierung

```
/* uv */ // uv is at offset 9 * bytes size(float)
glVertexArrayAttribFormat(VAO_Earth, 3, 2, GL_FLOAT, GL_FALSE, 9 * sizeof(float));
glEnableVertexArrayAttrib(VAO_Earth, 3);
glVertexArrayAttribBinding(VAO_Earth, 3, 0);
```

- Damit wird dem VAO mitgeteilt, dass die Texturkoordinaten an `layout(location=3)` liegen und im Vertex-Shader als `vec3` zugegriffen werden können.
 - Über `layout(location = 2) out vec3 out_UV` im Vertex-Shader werden diese an den Fragment-Shader an `location=2` mit `out_UV = in_UV` weitergeleitet.
-

4.4) Textur im Shader verwenden

Finden Sie nun den entsprechenden CPU-Seitigen Befehl, der die OpenGL-Textur, welche Sie in 4.4 erstellt haben, mit dem Shader verbindet.

Implementierung

```
glBindTextureUnit(0, textureHandle);
```

- Bindet Texture-Handler an Texture Unit0. Im Shader über uniform wird darauf zugegriffen.

Implementierung

- Im Fragment-Shader werden über die built-in Funktion `texture(sampler2D, in_UV.st)` die Texturfarben anhand der Texturkoordinaten aus der Textur (`u_Texture`) abgefragt.
- Der Sampler2D wurde in der Rendering-Loop mit `glBindTextureUnit(0, textureHandle)` an Texture Unit 0 gebunden, auf die im Shader über den uniform `sampler2D` zugegriffen werden kann.

Frage 1.:

Je nachdem, wie Sie Ihre Texturkoordinaten erstellt haben, sehen Sie das Bild auf dem Kopf. Wenn dem so ist, finden Sie eine Erklärung und einen Weg, dies zu ändern. Aber auch, wenn das Bild nicht auf dem Kopf steht, versuchen Sie zu verstehen, warum es passt. Nehmen Sie die Erkenntnisse in Ihre Dokumentation mit auf.

-> Lösung:

- Texturkoordinaten wurden so erstellt, dass die UV-Koordinaten von unten links (0, 0) nach oben rechts (1, 1) verlaufen. Die Textur steht aktuell auf dem Kopf. Wenn man aber (0, 0) auf oben links ändert und (1, 1) auf unten rechts, steht die Textur korrekt herum. Das liegt daran, dass

Bilddateien ihre Pixel standardmäßig von oben links nach unten rechts speichern, während OpenGL die UV-Koordinaten von unten links ausgehend interpretiert (siehe 4.1).

4.5) Texturkoordinaten für die Kugel erstellen

Nutzen Sie die Kugel, die Sie in task03 erstellt haben und weisen Sie dieser so Texturkoordinaten zu, sodass sich eine Weltkarte auf diese 'mappen' lässt. Rendern Sie die Kugel.

Implementierung

- Die Funktion *CreateSphere()* aus task03 generiert bereits Texturkoordinaten. Diese wurden so erstellt, dass die UV-Koordinaten von unten links (0, 0) nach oben rechts (1, 1) verlaufen. Dadurch wird die Weltkarte auf der Kugel korrekt dargestellt.
 - Die Textur der Weltkarte wird durch die zuvor erklärten Vertex- und Fragment-Shader auf die Fragments der Kugel gemappt.
-

4.6) Rotation der Kugel

Ermöglichen Sie die Rotation der Kugel mithilfe der Pfeiltasten, sodass Sie die Kugel von allen Blickwinkeln betrachten können.

Implementierung

```
// Reihenfolge für Rotation wichtig! Erst Welt-Y-Achse dann Welt-X-Achse
Mat4f earthModelMat = modelMat
    * Translate(Vec3f{ 0.0f, 0.0f, 0.0f })
    * baseScale
    * Rotate(RAMEN_WORLD_RIGHT, earthCameraAngleX)
    * Rotate(RAMEN_WORLD_UP, earthCameraAngleY)
;
```

- Wenn zuerst um die Welt-Y-Achse rotiert wird, dann um die Welt-X-Achse, bleibt die Rotation der Erde um ihre eigene Achse erhalten, auch wenn sie um die Welt-Y-Achse rotiert wird. Die Bewegung zu den Polen bleibt immer bestehen.
-

Ergebnis

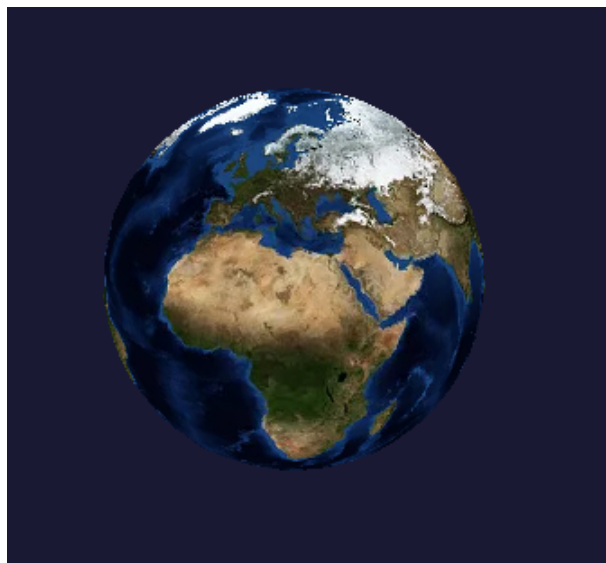


Figure 1: Kugel mit der Weltkarte als Textur.